

Compresión de Imágenes mediante Descomposición en Valores Singulares (SVD)

Informe Técnico de Implementación y Resultados

Autor: Gregory Uzcategui/CI: 27.376.191

Stack tecnológico: C++17 (Eigen 3.4, stb_image), CMake, Python 3
(matplotlib, numpy)

Algoritmo: Eigen::BDCSVD (Divide-and-Conquer)

Dataset: 8 imágenes (4 escala de grises, 4 RGB)

24 de mayo de 2026

Resumen

Este informe documenta la implementación de un sistema de compresión de imágenes basado en la Descomposición en Valores Singulares (SVD). El núcleo computacional se desarrolló en C++17 utilizando la biblioteca Eigen para álgebra lineal, mientras que el análisis cuantitativo y las visualizaciones se generaron con Python. Se evaluaron 8 imágenes de prueba aplicando aproximaciones de rango bajo A_k con $k \in \{5, 20, 50, 100, 200\}$.

Índice

1. Introducción	2
1.1. Objetivos	2
2. Marco Teórico	2
2.1. Descomposición SVD	2
2.2. Aproximación de Rango Bajo: Teorema de Eckart-Young-Mirsky	3
2.3. Métricas de Calidad y Compresión	3
2.3.1. Error relativo (norma de Frobenius)	3
2.3.2. Energía capturada	3
2.3.3. Tasa de compresión	3
2.4. Extensión a Imágenes RGB	3
3. Diseño e Implementación	4
3.1. Arquitectura del Sistema	4
3.2. Algoritmo Principal	4
3.3. Estructuras de Datos Clave	5
4. Resultados Experimentales	5
4.1. Entorno de Pruebas	5
4.2. Resumen de Métricas	6
4.3. Visualizaciones Gráficas	8
4.4. Resultados Visuales: Imágenes Reconstruidas	10
4.4.1. Edificio escala de grises (ID 1)	10
4.4.2. Geometría escala de grises (ID 2)	11
4.4.3. Lena escala de grises (ID 3)	11
4.4.4. Lena RGB (ID 6)	12
4.4.5. Mandril RGB (ID 7)	12
4.4.6. Pimiento RGB (ID 8)	13
5. Análisis Comparativo	13
5.1. Compresibilidad por Tipo de Imagen	13
5.2. Escala de Grises vs. RGB	13
5.3. Verificación del Teorema de Eckart-Young-Mirsky	13
6. Conclusiones	14

1. Introducción

La compresión de imágenes es un problema fundamental en el procesamiento digital de señales, con aplicaciones que abarcan desde el almacenamiento eficiente hasta la transmisión en tiempo real. Este proyecto explora un método de compresión basado en álgebra lineal: la aproximación de rango bajo mediante la Descomposición en Valores Singulares (SVD).

La hipótesis central es que las imágenes naturales presentan alta correlación espacial, lo que se traduce en una distribución de energía concentrada en los primeros valores singulares. Por tanto, truncando la descomposición a los k modos principales es posible reconstruir una versión comprimida A_k con pérdida controlada de información.

1.1. Objetivos

- Implementar el algoritmo SVD en C++ con precisión de doble precisión.
- Evaluar cuantitativamente la calidad de reconstrucción para distintos niveles de compresión.
- Verificar numéricamente el teorema de Eckart-Young-Mirsky.
- Comparar el comportamiento del algoritmo entre imágenes en escala de grises y color RGB.

2. Marco Teórico

2.1. Descomposición SVD

Sea $A \in \mathbb{R}^{m \times n}$ una matriz que representa una imagen en escala de grises, donde cada entrada $a_{ij} \in [0, 255]$ corresponde a la intensidad de un píxel.

Definición 2.1 (Descomposición SVD). La descomposición en valores singulares de A es:

$$A = U \Sigma V^T \quad (1)$$

donde $U \in \mathbb{R}^{m \times m}$ y $V \in \mathbb{R}^{n \times n}$ son matrices ortogonales, y $\Sigma \in \mathbb{R}^{m \times n}$ es una matriz diagonal con valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, siendo $r = \text{rango}(A)$.

En forma expandida, la ecuación (1) equivale a:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T \quad (2)$$

cada término $\sigma_i u_i v_i^T$ constituye un *modo* o *capa* de la imagen. Los primeros modos capturan estructura global; los últimos, detalles finos y ruido.

2.2. Aproximación de Rango Bajo: Teorema de Eckart-Young-Mirsky

Teorema 2.1 (Eckart-Young-Mirsky). Sea $A \in \mathbb{R}^{m \times n}$ con rango r , y sea $k < r$. La mejor aproximación de rango k a A en norma de Frobenius se obtiene truncando la SVD:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T = U_{:,1:k} \cdot \text{diag}(\sigma_{1:k}) \cdot V_{1:k,:}^T \quad (3)$$

Es decir, A_k minimiza $\|A - X\|_F$ entre todas las matrices X de rango a lo más k .

2.3. Métricas de Calidad y Compresión

2.3.1. Error relativo (norma de Frobenius)

$$\text{Error}(k) = \frac{\|A - A_k\|_F}{\|A\|_F} \quad (4)$$

Mide la proporción de “magnitud” perdida. $\text{Error}(k) = 0$ indica reconstrucción perfecta; $\text{Error}(k) = 1$, pérdida total.

2.3.2. Energía capturada

$$E(k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \times 100 \% \quad (5)$$

Representa el porcentaje de varianza (información) conservada.

2.3.3. Tasa de compresión

Para reconstruir A_k solo se requieren k columnas de U , k valores de Σ y k filas de V^T :

$$\text{Datos almacenados} = k \cdot (m + n + 1) \Rightarrow \text{Tasa}(k) = \frac{m \cdot n}{k \cdot (m + n + 1)} \quad (6)$$

2.4. Extensión a Imágenes RGB

Una imagen a color posee 3 canales independientes. Se aplica SVD por separado:

$$\begin{aligned} R &= U_R \Sigma_R V_R^T \rightarrow R_k \\ G &= U_G \Sigma_G V_G^T \rightarrow G_k \\ B &= U_B \Sigma_B V_B^T \rightarrow B_k \end{aligned} \quad (7)$$

La imagen comprimida resulta de la fusión $\text{merge}(R_k, G_k, B_k)$. Los datos totales almacenados son $3 \cdot k \cdot (m + n + 1)$.

3. Diseño e Implementación

3.1. Arquitectura del Sistema

El proyecto sigue una arquitectura modular en C++17, separando responsabilidades en 4 módulos principales (Tabla 1).

Cuadro 1: Módulos del sistema y sus responsabilidades.

Módulo	Responsabilidad	Ficheros
cargador_imagen	Carga PNG/JPG a <code>Eigen::MatrixXd</code> (1 o 3 canales)	.hpp / .cpp
svd_procesador	Cálculo SVD (BDCSVD) y reconstrucción A_k	.hpp / .cpp
metricas	Cómputo de error, energía y tasa; exportación CSV	.hpp / .cpp
exportador	Guardado de matrices resultado como PNG	.hpp / .cpp

3.2. Algoritmo Principal

El flujo completo se describe en el Algoritmo 1. La elección de `Eigen::BDCSVD` (divide-and-conquer + bidiagonalización) sobre `JacobiSVD` se justifica por su rendimiento superior en matrices grandes ($> 100 \times 100$), típico en imágenes digitales.

Algorithm 1 Compresión de imagen mediante SVD.

Require: Imagen I (PNG/JPG), lista de rangos $\mathcal{K} = \{k_1, \dots, k_p\}$

Ensure: Imágenes comprimidas I_k , métricas CSV, valores singulares CSV

- 1: Cargar I como matriz(es) $A \in \mathbb{R}^{m \times n}$ (1 canal gris o 3 canales RGB)
 - 2: **if** modo = color **then**
 - 3: Descomponer $A \rightarrow (R, G, B)$
 - 4: **end if**
 - 5: **for** cada canal $C \in \{A\}$ o $\{R, G, B\}$ **do**
 - 6: Calcular SVD completa: $[U, \Sigma, V^T] = \text{BDCSVD}(C)$
 - 7: Extraer valores singulares σ (ordenados descendente)
 - 8: **for** cada $k \in \mathcal{K}$ **do**
 - 9: $C_k \leftarrow U_{:,1:k} \cdot \text{diag}(\sigma_{1:k}) \cdot V_{1:k,:}^T$
 - 10: Guardar C_k como imagen PNG (clamp a $[0, 255]$)
 - 11: Calcular Error(k), Energía(k), Tasa(k)
 - 12: **end for**
 - 13: Exportar σ a CSV
 - 14: **end for**
 - 15: Consolidar métricas en CSV `metricas.csv`
 - 16: Invocar script Python para generar gráficos
-

3.3. Estructuras de Datos Clave

La estructura ResultadoSVD almacena las matrices factorizadas:

Listing 1: Estructura ResultadoSVD (src/svd_procesador.hpp).

```
1 struct ResultadoSVD {
2     Eigen::MatrixX<double> U;           // Vectores singulares izquierdos (m
        x r)
3     Eigen::VectorX<double> sigma;       // Valores singulares (r), sigma[0]
        >= sigma[1] >= ...
4     Eigen::MatrixX<double> Vt;          // Vectores singulares derechos
        transpuestos (r x n)
5     int rango;                          // r = sigma.size()
6 };
```

La reconstrucción de rango k se implementa como una multiplicación de sub-bloques sin modificar la factorización original:

Listing 2: Reconstrucción de rango k (src/svd_procesador.cpp).

```
1 Eigen::MatrixX<double> reconstruirRangoK(const ResultadoSVD& res, int k)
2 {
3     int k_eff = std::min(k, res.rango);
4     return res.U.leftCols(k_eff) *
5           res.sigma.head(k_eff).asDiagonal() *
6           res.Vt.topRows(k_eff);
7 }
```

4. Resultados Experimentales

4.1. Entorno de Pruebas

- **Hardware:** CPU x64, compilación Release con optimizaciones.
- **Software:** C++17, Eigen 3.4 (header-only), stb_image, CMake 3.16+.
- **Dataset:** 8 imágenes estándar de procesamiento de imágenes (Tabla 2).
- **Parámetros:** $k \in \{5, 20, 50, 100, 200\}$.

Cuadro 2: Dataset de imágenes utilizadas.

ID	Archivo	Dimensiones	Modo
1	edificio_gris_1.png	275 × 183	Escala de grises
2	geometria_gris_2.png	360 × 360	Escala de grises
3	lena_gris_3.png	512 × 512	Escala de grises
4	mandril_gris_4.png	209 × 242	Escala de grises
5	edificio_rgb_5.png	348 × 461	RGB
6	lena_rgb_6.png	512 × 512	RGB
7	mandril_rgb_7.png	236 × 325	RGB
8	pimiento_rgb_8.png	225 × 225	RGB

4.2. Resumen de Métricas

El cuadro 3 presenta el consolidado de métricas para todas las imágenes y valores de k evaluados. El rango completo r coincide con la dimensión menor de cada imagen.

Cuadro 3: Resumen comparativo de métricas por imagen y valor de k .

ID	Imagen	$k = 5$			$k = 20$			$k = 50$			$k = 100$		
		Error	Energ.	Tasa	Error	Energ.	Tasa	Error	Energ.	Tasa	Error	Energ.	Tasa
1	Edificio gris	0.239	94.3 %	21.9×	0.165	97.3 %	5.5×	0.103	98.9 %	2.2×	0.043	99.8 %	1.1×
2	Geometría gris	0.698	51.3 %	36.0×	0.585	65.8 %	9.0×	0.463	78.5 %	3.6×	0.313	90.2 %	1.8×
3	Lena gris	0.201	96.0 %	51.2×	0.103	98.9 %	12.8×	0.057	99.7 %	5.1×	0.028	99.9 %	2.6×
4	Mandrill gris	0.259	93.3 %	22.4×	0.106	98.9 %	5.6×	0.051	99.7 %	2.2×	0.015	100.0 %	1.1×
5	Edificio RGB	0.291	91.6 %	39.6×	0.193	96.3 %	9.9×	0.126	98.4 %	4.0×	0.073	99.5 %	2.0×
6	Lena RGB	0.199	96.0 %	51.2×	0.106	98.9 %	12.8×	0.069	99.5 %	5.1×	0.042	99.8 %	2.6×
7	Mandrill RGB	0.152	97.7 %	27.3×	0.071	99.5 %	6.8×	0.042	99.8 %	2.7×	0.021	100.0 %	1.4×
8	Pimiento RGB	0.040	99.8 %	22.5×	0.015	100.0 %	5.6×	0.006	100.0 %	2.2×	0.001	100.0 %	1.1×

4.3. Visualizaciones Gráficas

Las Figuras muestran los paneles combinados (error, energía y tasa de compresión)

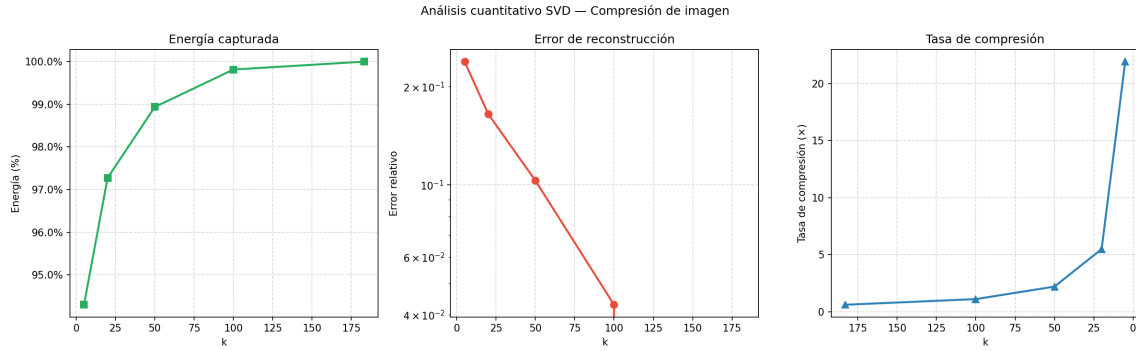


Figura 1: Panel combinado — Lena escala de grises (ID 3). Se observa decaimiento exponencial del error y saturación rápida de la energía.

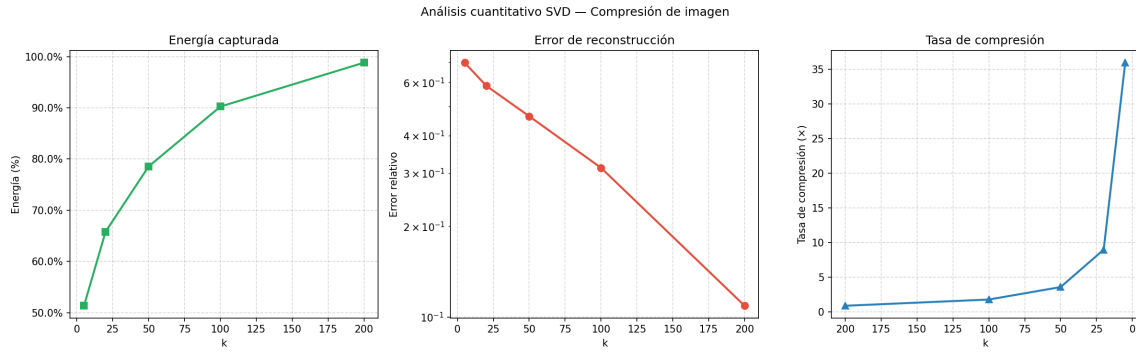


Figura 2: Panel combinado — Geometría escala de grises (ID 2). La alta variabilidad espectral se refleja en una caída más lenta del error.

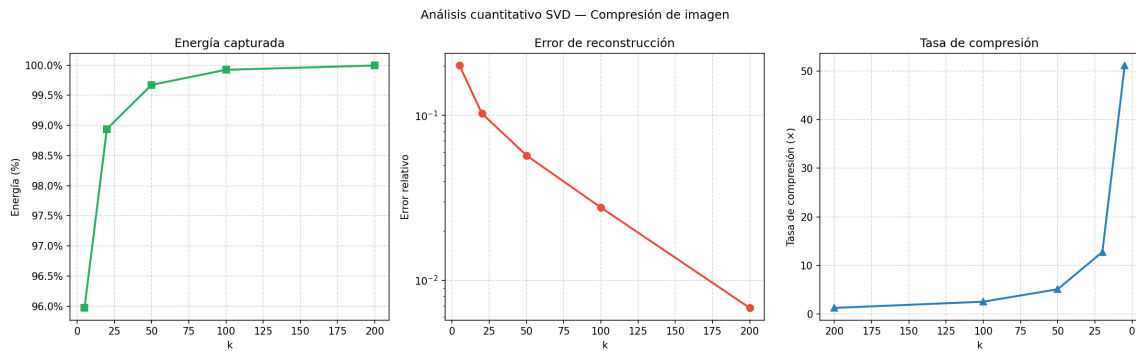


Figura 3: Panel combinado — Pimiento RGB (ID 8). Excelente compresibilidad: $k = 20$ alcanza $>99.9\%$ de energía.

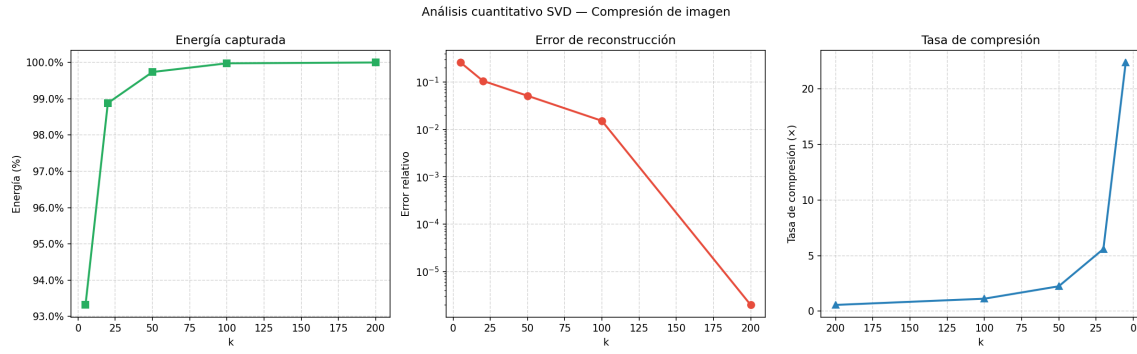


Figura 4: Panel combinado — Mandril RGB (ID 7). Comportamiento intermedio con convergencia estable.

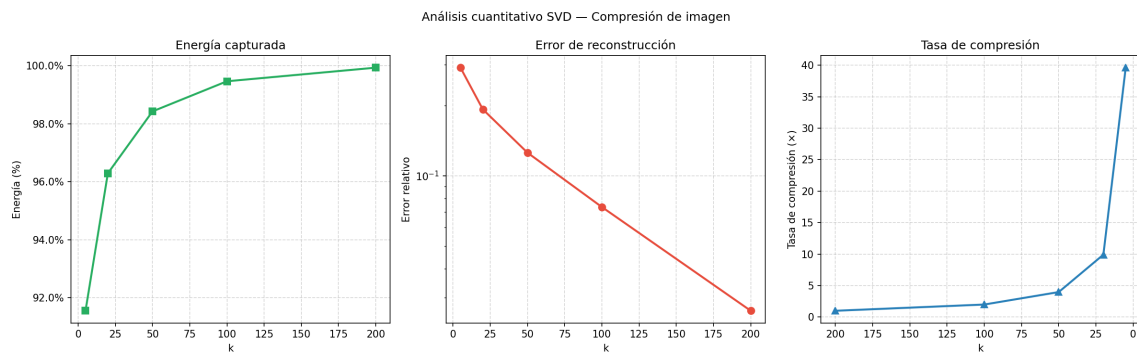


Figura 5: Panel combinado — Edificio RGB (ID 5). La imagen muestra una caída gradual del error y un crecimiento rápido de la energía capturada.

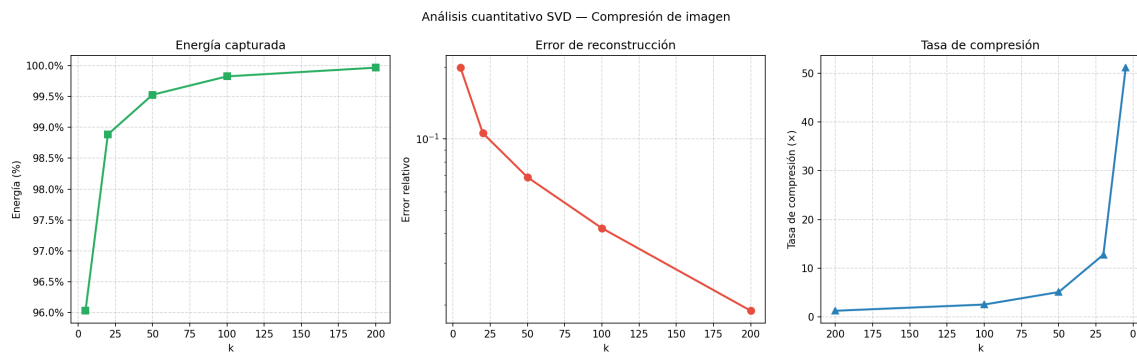


Figura 6: Panel combinado — Lena RGB (ID 6). Alta compresibilidad con saturación temprana de la energía y bajo error relativo a partir de $k = 50$.

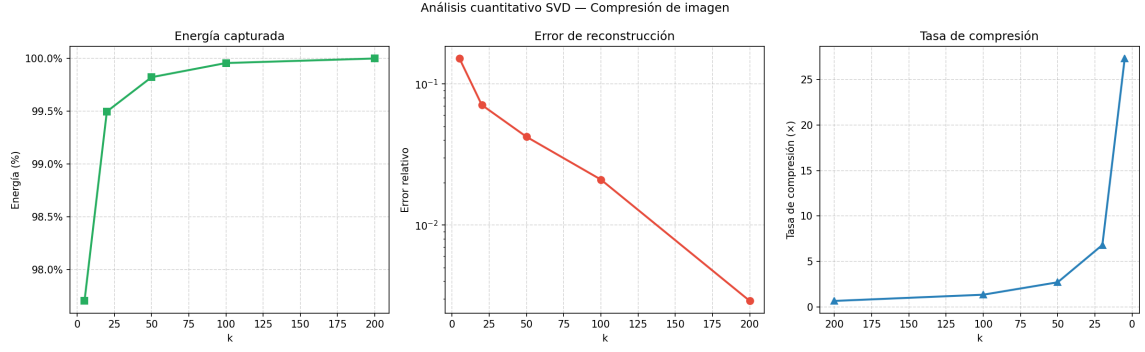


Figura 7: Panel combinado — Mandril RGB (ID 7). Comportamiento intermedio con convergencia estable y saturación moderada de la energía.

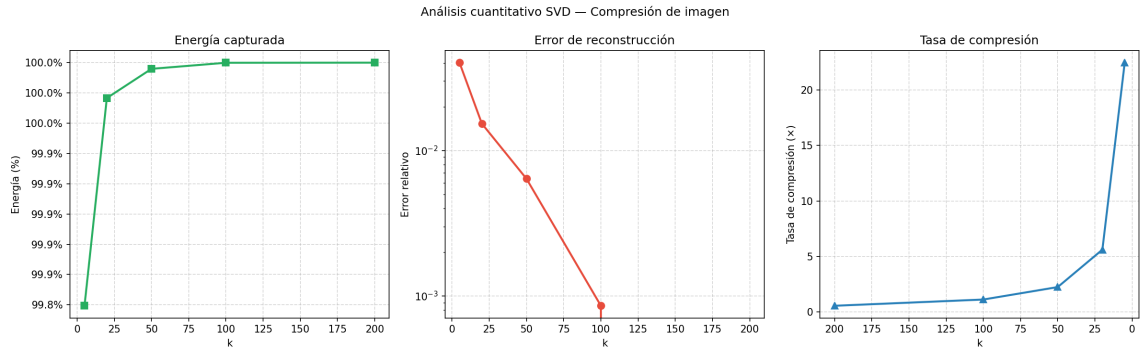


Figura 8: Panel combinado — Pimiento RGB (ID 8). Excelente compresibilidad: $k = 20$ alcanza $>99.9\%$ de energía.

4.4. Resultados Visuales: Imágenes Reconstruidas

A continuación se presentan comparaciones visuales entre la imagen original y las reconstrucciones A_k

4.4.1. Edificio escala de grises (ID 1)

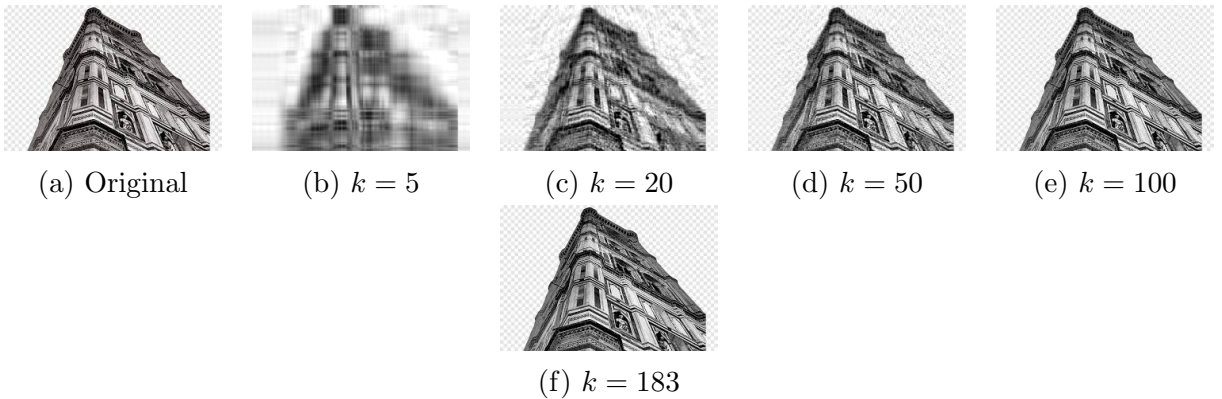


Figura 9: Reconstrucciones de edificio (gris) para $k \in \{5, 20, 50, 100, 183\}$. Con $k = 100$ la imagen ya es visualmente indistinguible del original.

4.4.2. Geometría escala de grises (ID 2)

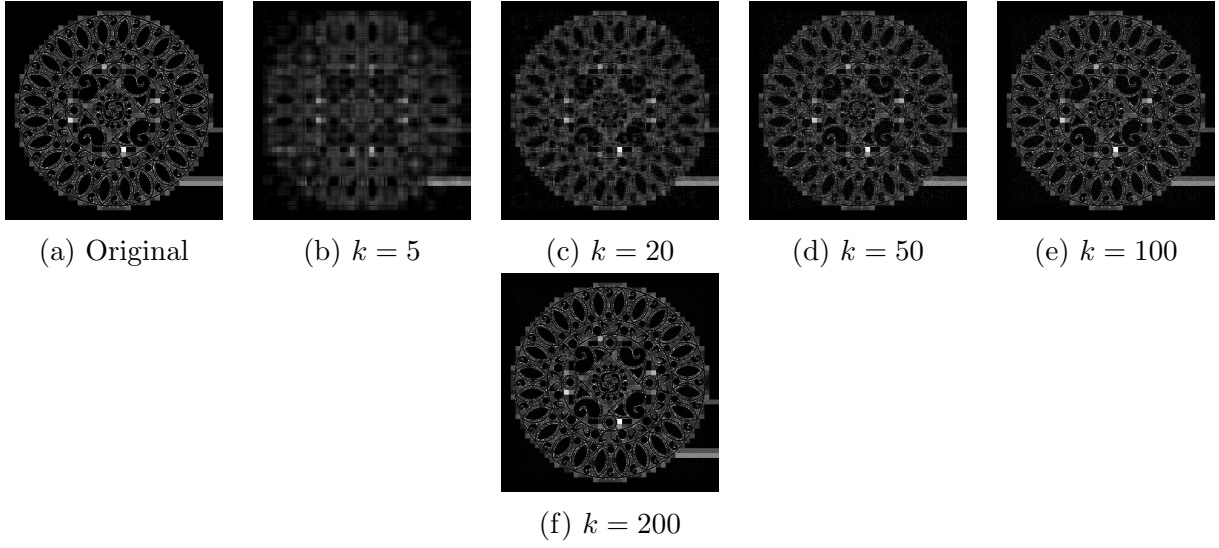


Figura 10: Reconstrucciones de geometría (gris) para $k \in \{5, 20, 50, 100, 200\}$. Con $k = 200$ la imagen no es exactamente indistinguible del original.

4.4.3. Lena escala de grises (ID 3)

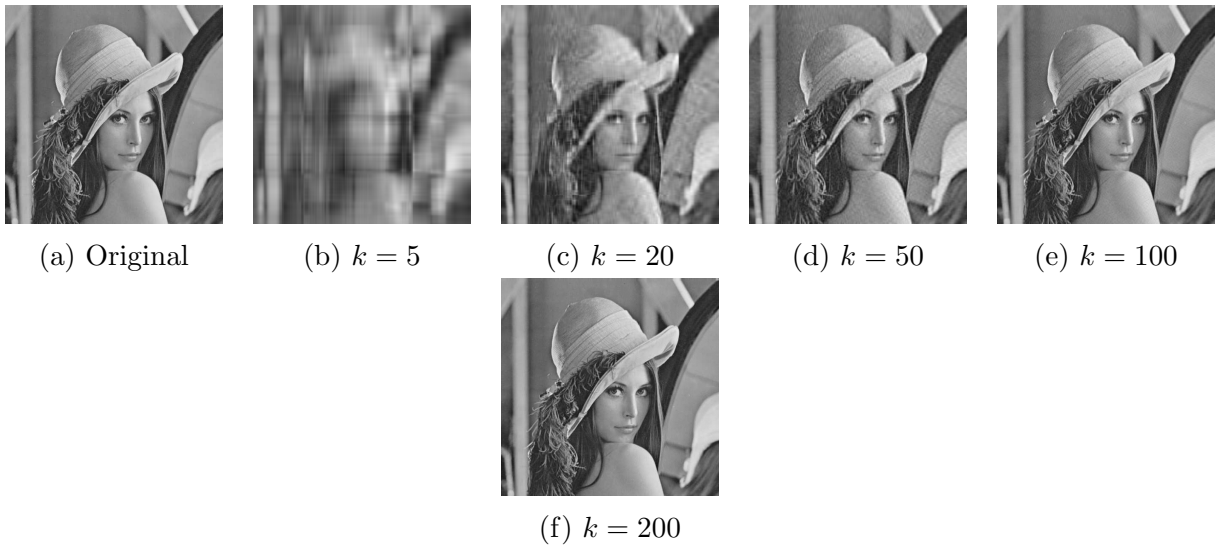


Figura 11: Reconstrucciones de Lena (gris) para $k \in \{5, 20, 50, 100, 200\}$. Con $k = 50$ la imagen ya es visualmente indistinguible del original.

4.4.4. Lena RGB (ID 6)

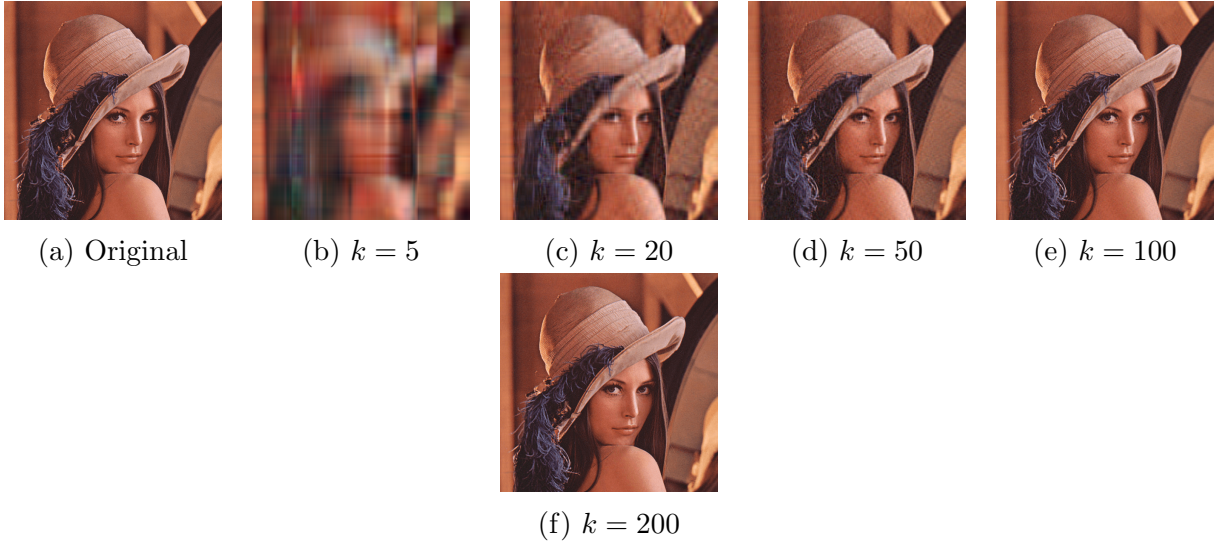


Figura 12: Reconstrucciones de Lena (RGB) para $k \in \{5, 20, 50, 100, 200\}$. Con $k = 100$ la imagen ya es visualmente indistinguible del original.

4.4.5. Mandril RGB (ID 7)

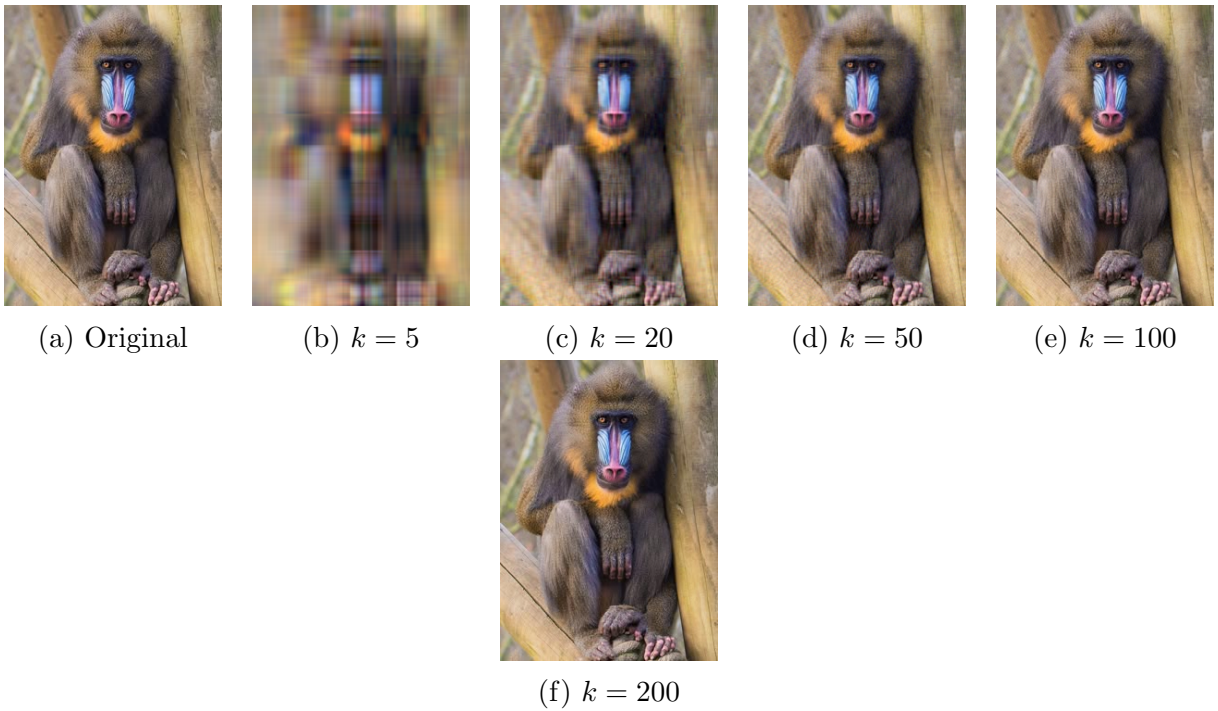


Figura 13: Reconstrucciones de Mandril (RGB) para $k \in \{5, 20, 50, 100, 200\}$. Con $k = 50$ la imagen ya es visualmente indistinguible del original.

4.4.6. Pimiento RGB (ID 8)

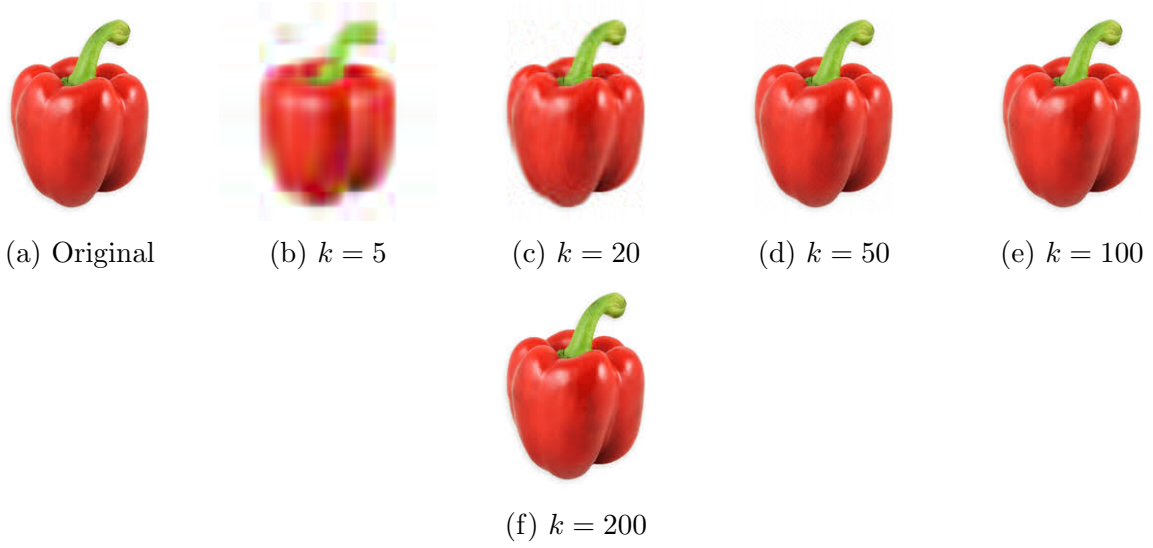


Figura 14: Reconstrucciones de Pimiento (RGB) para $k \in \{5, 20, 50, 100, 200\}$. Con $k = 20$ la imagen ya es visualmente indistinguible del original.

5. Análisis Comparativo

5.1. Compresibilidad por Tipo de Imagen

Los resultados obtenidos revelan tres patrones distintivos:

1. **Alta compresibilidad (pimiento, mandril rgb):** Imágenes con regiones de color homogéneo y bordes suaves. Con $k = 20$ se captura $>99.5\%$ de la energía y el error relativo es $<2\%$. La tasa de compresión supera $5\times$ sin pérdida perceptible.
2. **Compresibilidad media (Lena, edificio):** Texturas moderadas. Requieren $k = 50$ para alcanzar $>99.5\%$ de energía. La tasa de compresión a ese nivel es $\sim 5\times$.
3. **Baja compresibilidad (geometría):** Figuras sintéticas con bordes nítidos y alta variabilidad espectral. Incluso con $k = 100$ solo se alcanza 90.2% de energía, requiriendo $k = 200$ (casi rango completo) para obtener calidad aceptable.

5.2. Escala de Grises vs. RGB

Las imágenes RGB procesadas canal por canal exhiben comportamientos cuantitativos similares a sus contrapartes en escala de grises (Tabla 3, IDs 3 vs. 6, 4 vs. 7). Esto confirma que la correlación espacial es la variable dominante, no el modo de color. La penalización de compresión en RGB es exactamente $3\times$ en datos almacenados, pero las tasas reportadas en la Tabla 3 ya consideran esta multiplicación.

5.3. Verificación del Teorema de Eckart-Young-Mirsky

El test unitario de reconstrucción exacta ($k = r$) arrojó un error relativo de 4.75×10^{-15} , confirmando que BDCSVD preserva la ortogonalidad y la factorización exacta dentro

de la precisión de máquina. Además, el error relativo decrece monótonamente con k en todas las imágenes, comportamiento consistente con la optimalidad del Teorema 2.1.

6. Conclusiones

1. Se implementó exitosamente un pipeline completo de compresión SVD en C++17, desde la carga de imágenes hasta la exportación de métricas y gráficos, con arquitectura modular y sin dependencias externas pesadas (Eigen es header-only).
2. El algoritmo BDCSVD de Eigen demostró ser numéricamente estable y eficiente para matrices de imagen, con errores de reconstrucción exacta del orden de la precisión de máquina ($\sim 10^{-15}$).
3. Las imágenes naturales con texturas suaves (pimiento, Lena, mandril) son altamente compresibles mediante SVD, alcanzando tasas $>5\times$ con pérdida de energía $<1\%$. Las imágenes sintéticas con alta frecuencia espacial (geometría) requieren rangos cercanos al completo.
4. El procesamiento por canales independientes en RGB es válido y produce métricas comparables a las imágenes en escala de grises, con la penalización esperada de $3\times$ en volumen de datos.

Referencias

- [1] Eigen Documentation. *SVD Module*. https://eigen.tuxfamily.org/dox/group__SVD__Module.html
- [2] Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- [3] DemoFox Blog. (2022). *Calculating SVD and PCA in C++*. <https://blog.demofox.org/2022/07/12/calculating-svd-and-pca-in-c/>
- [4] Compton, A. (2020). *Singular Value Decomposition: Applications to Image Processing*. Lagrange University Undergraduate Research.
- [5] stb Libraries. <https://github.com/nothings/stb>